

Software Testing

Testing approach

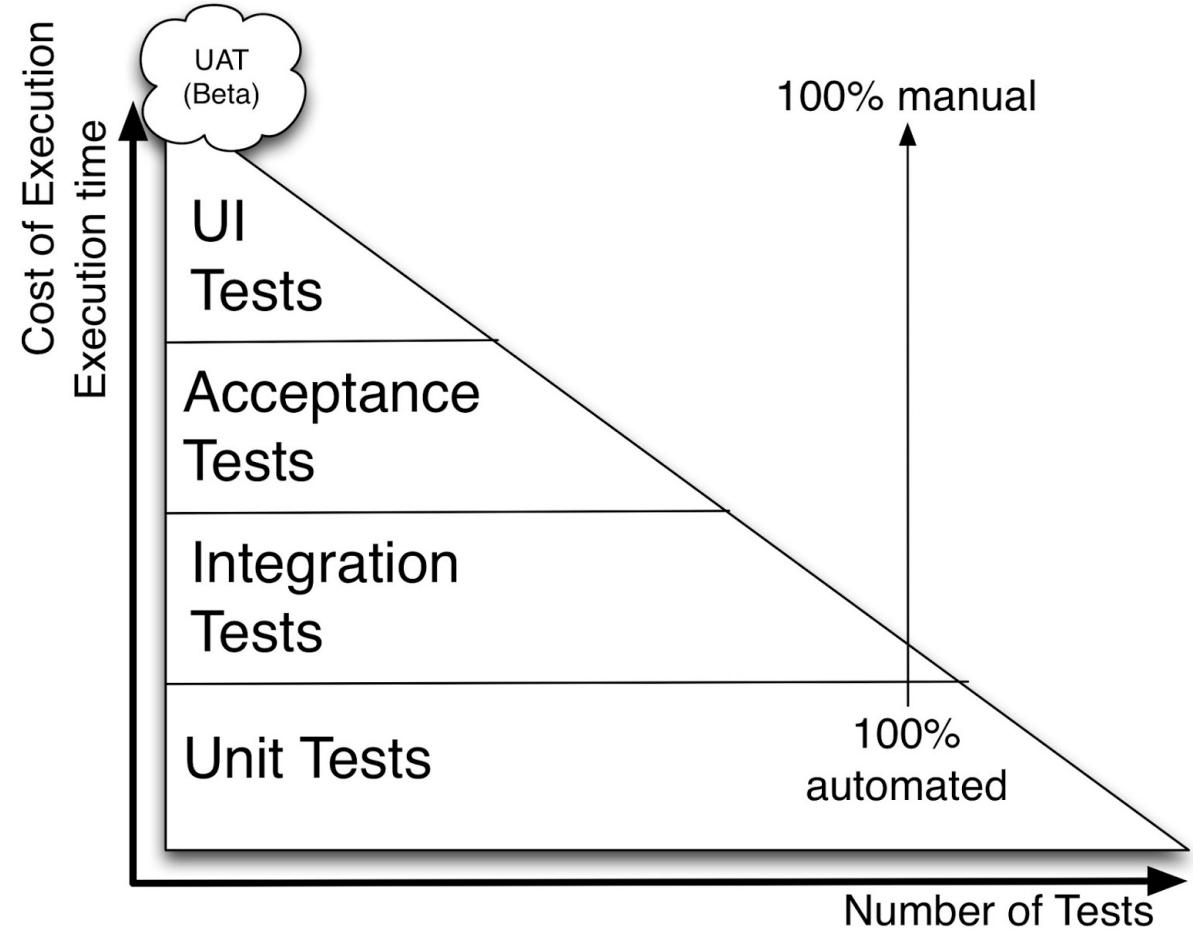
According to IEEE Std 829-1998,

Software testing is defined as

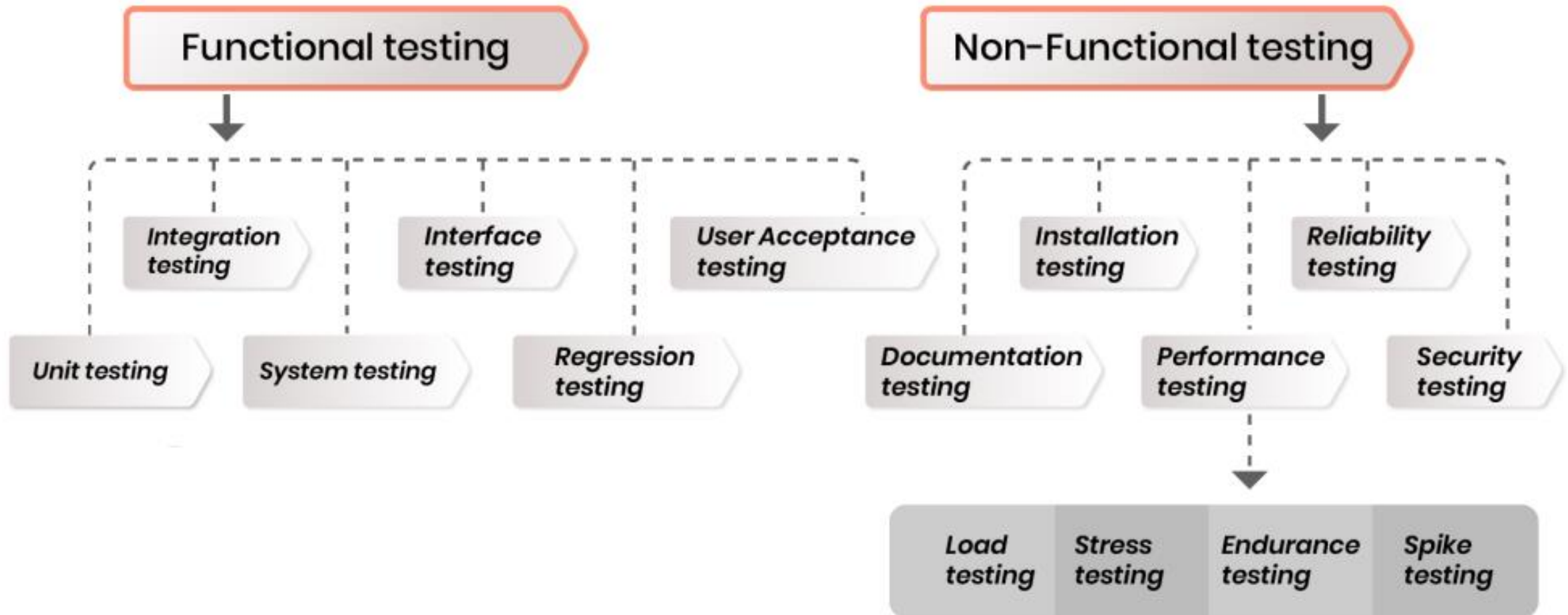
“the process of analyzing a software item, to detect the differences between existing and required conditions (defects) and evaluating the features of the software items.”

Testing level

- Unit (Level 0)
- Integration
- System
- Acceptance
- User Interface (Level n)



Types of testing



Software Testing

- Strategies & Tactics
- Blackbox (requirements) & Whitebox (inner system) testing.
- Component level testing.
 - Components, modules, classes, interfaces (units)...
- Object-oriented testing.
 - Class
 - Behavior
- Integration level testing
 - Top-Down & Bottom-Up integration.
 - Continuous Integration.
 - AI Test integration.
 - In OO Context
 - Fault-Based
 - Scenario-Based (use-cases)

“You test quality, if it isn’t there before you start testing,
it want be there when you’re done testing.”

Testing is not a fix for quality, at the end of the process.

From book

- **Conflict between** the proud **developer(s) and tester** that want to break it down and tear it apart to find the errors.
- Criteria of Done = Acceptance criteria (Given-When-Then)
- When are we done testing?
 - You're never done testing, the burden has just shifted to the user.
 - Every time a user uses the system, it's being tested.
 - You're done testing, when you run out of time or money.

Best Testing Strategy

Best test strategy

Is when the software testers,

- **Specify product requirements in a quantifiable manner**, long before testing begins.
- **State testing objectives** explicitly.
- **Understands the users of the system** and develop a profile for each user category.
- **Develops a testing plan** that emphasizes “Rapid cycle testing”.
- **Build “robust” software** that is designed to test itself, use effective technical reviews as a filter prior to testing.
- **Conduct technical reviews** to assess the test strategy and test cases.
- **Develop a continuous improvement approach.**

Best test strategy

- Test planning.
- Test case design.
- Test execution.
- Test result data collection and evaluation.

In agile

- **Test plans** needs to be **established** before the first sprint meeting **and reviewed** by stakeholders
 - Rough timeline, standards and testing tools to be used.
- **Testing cases and directions** on their use are developed and reviewed by stakeholders when the relating use story gets mature for a sprint
 - Using ex. Given-When-Then acceptance criteria to express when a story is acceptable (fulfills its purpose acceptably).
- **Striving to make rapid and transparent test results**, so everyone knows the state of the software.

Test recordkeeping

- **Test cases can be recorded in a simple format**, in Google Sheets, and linked to in a relating story in Jira or another agile project tool
- Containing
 - **Acceptance criteria** for success like Given-When-Then
 - **Test cases** with pointer / links to related requirements.
 - **Expected in- and output** from the test cases.
 - **Allowing the testers to indicate**
 - Whether the test passed or failed.
 - Dates for when test case was run.
 - Comments about observations and assumptions to validate.

Generic Test Characteristics

- Effective testing needs technical reviews, to find errors before the test phase commences.
- Testing begins at the component level and works “outward” towards the integration of the entire system.
- Different testing techniques are appropriate for different engineering approaches and at different points in time.
- Testing is done by the developer of the system and in large projects, a independent test-team can be assigned.

Verification and Validation (V&V)

- **Verification** refers to a set of tasks that ensure that software correctly implements a specific function.

Are we building the product right?

- **Validation** refers to a different set of tasks that ensure that the software that has been built is traceable customer requirements.

Are we building the right product?

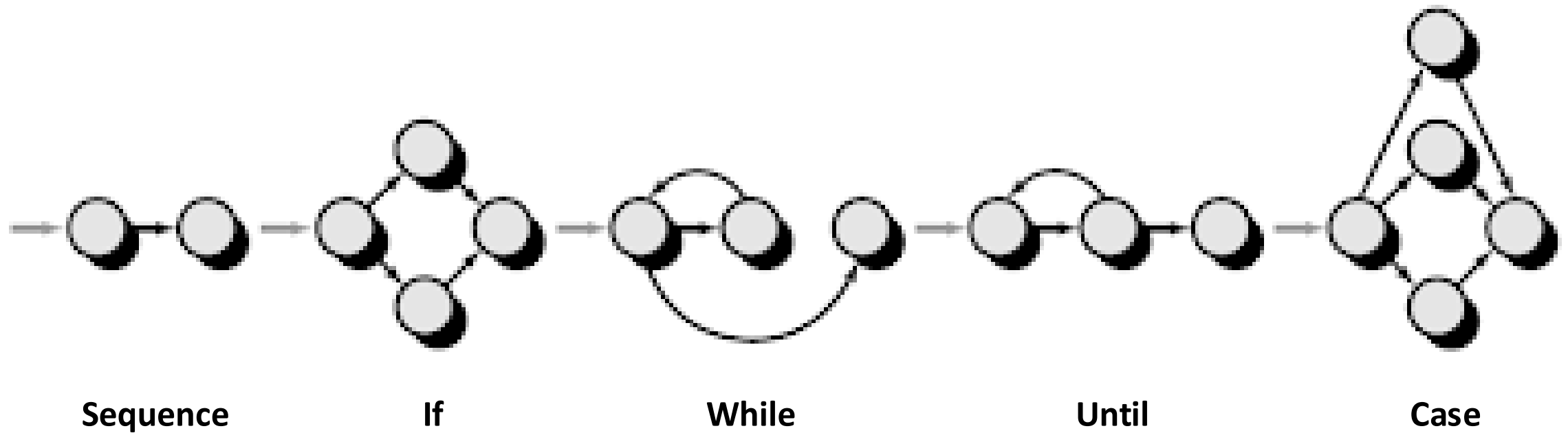
Verification- and Validation activities

- Technical reviews.
- Quality audits.
- Configuration audits.
- Performance monitoring.
- Simulation.
- Feasibility studies.
- Documentation review.
- Database review.
- Algorithm analysis.
- Development testing.
- Usability testing.
- Qualification testing.
- Acceptance testing.
- Installation testing.

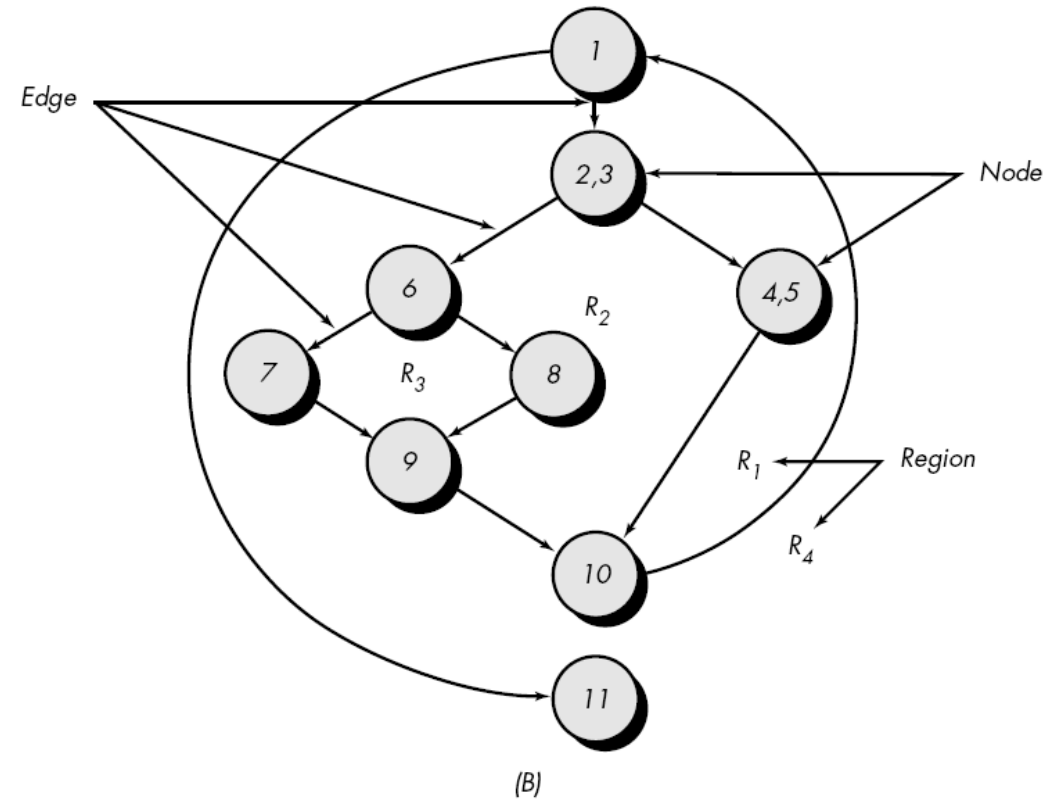
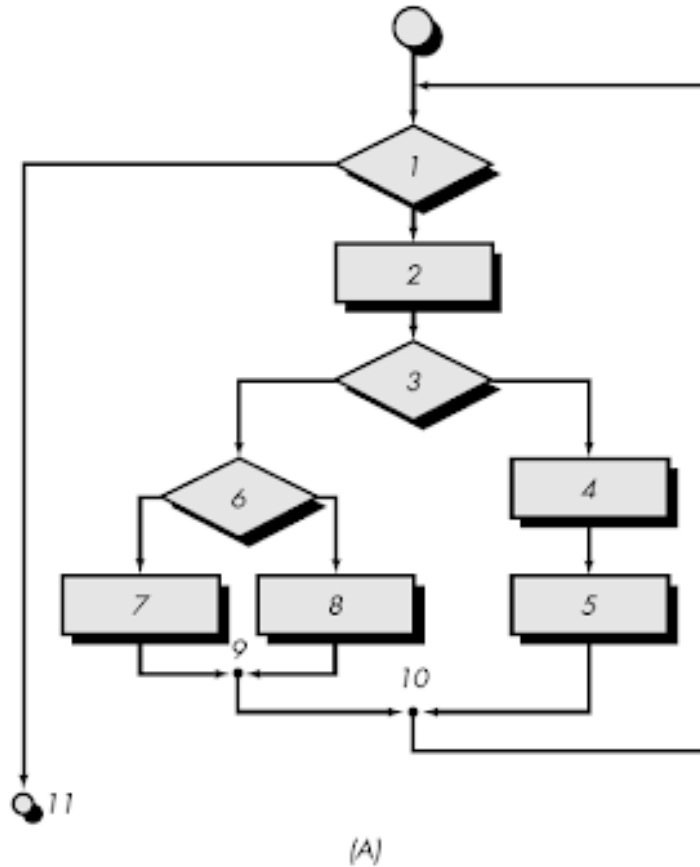
White Box Testing (inside the system)

- The Basis Path.
- Control Structure Testing.

The Basis Path



The Basis Path



path 1: 1-11

path 2: 1-2-3-4-5-10-1-11

path 3: 1-2-3-6-8-9-10-1-11

path 4: 1-2-3-6-7-9-10-1-11

Cyclomatic complexity

- A **quantitative measure** of the **logical complexity** of a program.
- Defines the number of **independent paths** in a program context.
- The **upper bound** for the **number of tests that must be conducted** to ensure that all statements have been executed at least once.
- An **independent path** is any path that **introduces** at least one **new set of processing statements** or a **new condition**.

Cyclomatic complexity

Computed in 3 ways:

#1

The number of regions.

#2

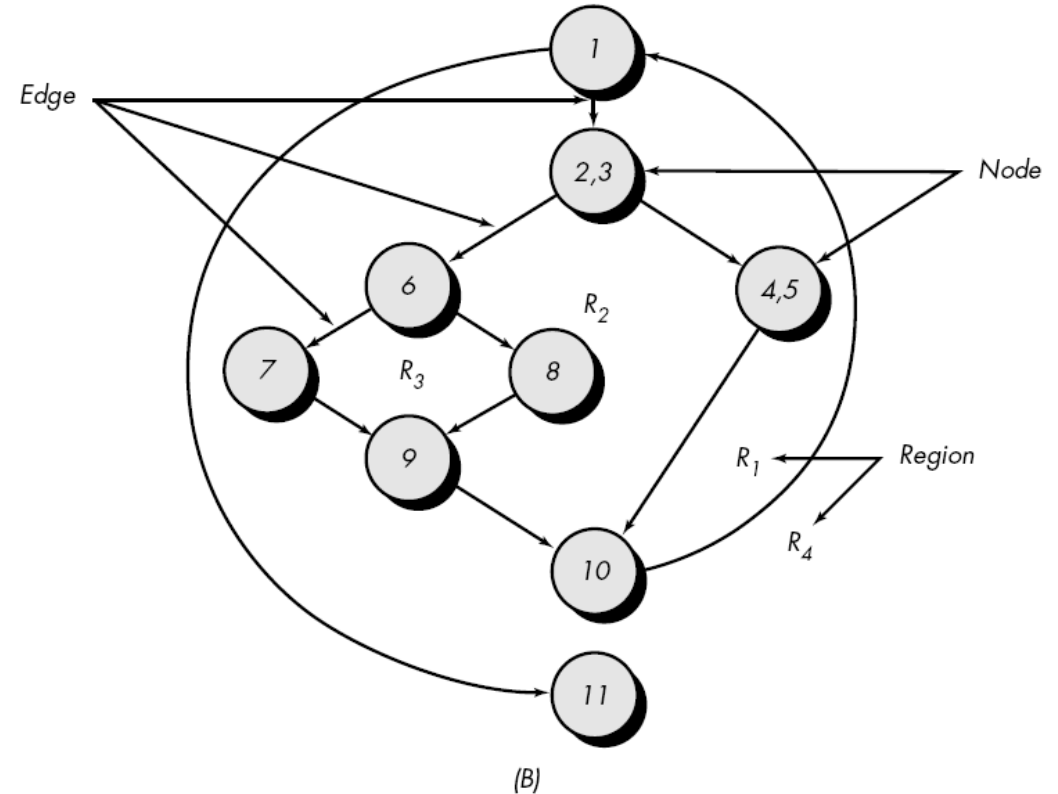
$$V(G) = E - N + 2$$

where E is the **number edges**, N is the **number of nodes**.

#3

$$V(G) = P + 1$$

where P is the **number of predicate nodes**.



Cyclomatic complexity

computed using each of the algorithms just noted:

The flow graph has four regions.

$$V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4.$$

$$V(G) = 3 \text{ predicate nodes} + 1 = 4.$$

Control Structure Testing

- Condition testing.
- Dataflow testing.
- Loop testing.

Black Box Testing (requirements)

- Interface testing
 - Assuring the component interfaces adhere to the expected use-cases.
- Equivalence partitioning
 - Splitting tests up into different classes of input data domains, that test cases can be derived from.
- Boundary Value Analysis (BVA)
 - We test values inside and outside the value boundaries, but not all possible values.
 - A **range bounded by values (A,B)**, then test cases should be designed with values A and B and just above and below A and B.
 - A sequence of **number of values (1..100)**, should exercise minimum and maximum values + values just above and below minimum and maximum.

Object-Oriented Testing

- Class testing.
- Behavior testing
 - Traversing
 - Breadth-First.
 - Depth-First.
 - Path via pre-defined states.

Functional Tests

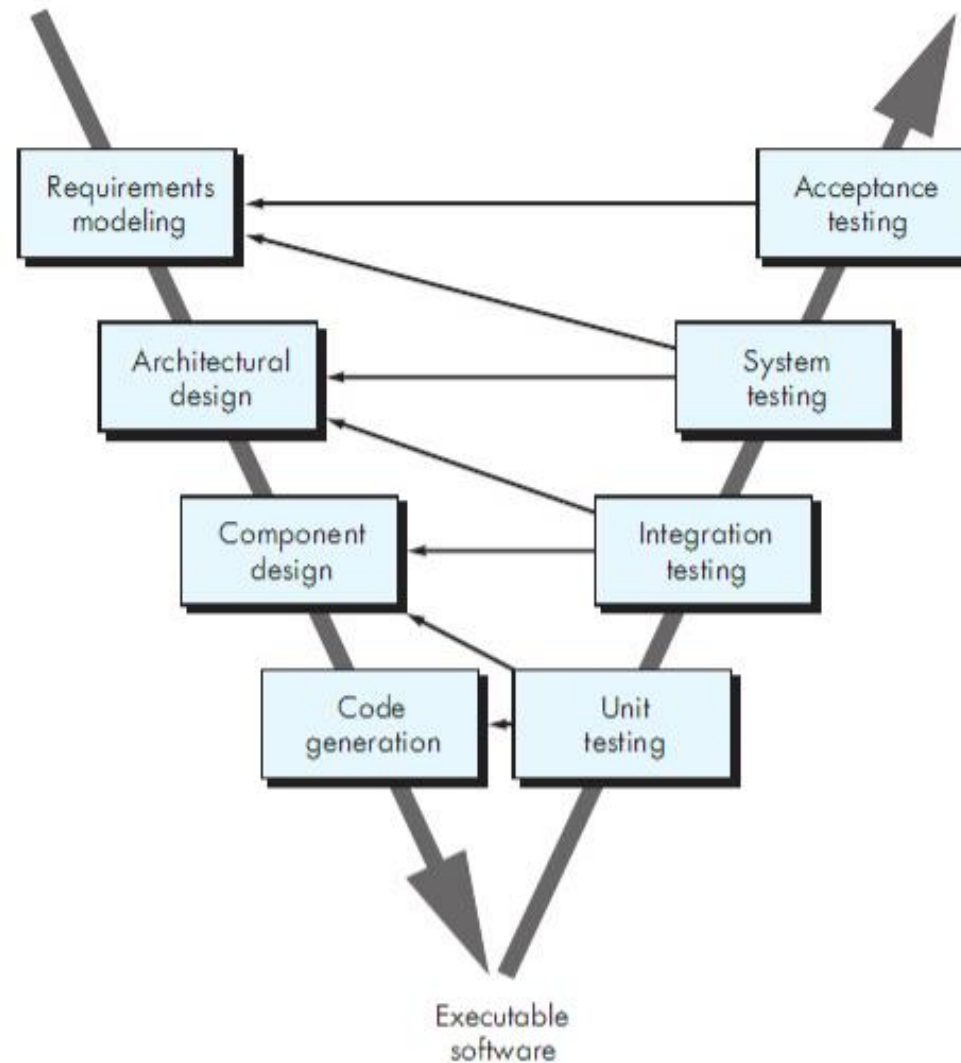
Functional Tests

- Unit
- Integration
- Interface
- System
- Regression
- Smoke
- Sanity
- User Acceptance
- End-to-end
- Browser
- White box
- Black box

The V Model

FIGURE 4.2

The V-model



Unit Tests

- The most basic unit or component of an application is tested.
- Usually done by the developer to ensure the smallest testable code is working fine.
- It is a White Box testing technique.
- V-Model location
 - In the bottom.
 - It is the first testing type to be carried out .

Integration Tests

- When two or more components or units are integrated together, they need to interact with each other in the form of commands, data exchange or database calls.
- Integration testing is performed on them as a single cluster to check that the interaction between them is happening, as expected.

Interface Tests

- Part of Integration testing.
- It comes under, the correctness of data exchange or transfer between two components is tested in Interface testing.
- Ex. One component creates a .csv file as output, and another component processes the .csv file into XML and sends to the third component.
- During this data transfer, the data should remain intact, and all the components should be able to process the file and send the file to the next component successfully.

System Tests

- All the modules of the application are combined and the whole system is tested as a single unit for correctness against the requirement specification.

Regression Tests

- Whenever there are some code fixes, or any enhancement, the code is modified.
- Makes sure new code changes have not injected any new defects into the code and that the previously working functionality is still intact and working.

Smoke Tests

- Performed on initial unstable builds, whenever a new build is released by developers, testing team performs Smoke testing to be sure that all end-to-end functionalities are working.
- If any of the major functionality is broken due to a new build, the build is rejected and sent back to the developers.
- Performed to ensure new code changes have not broken any major functionality and the build can be taken forward for the next level of testing.

Sanity Tests

- Part of Regression testing.
- Performed when a new build is released for a stable application.
- Only after running the Sanity test suite, will the build be taken forward for the next level of testing.
- Difference between Smoke and Sanity is
 - Smoke testing is performed on an initial unstable application
 - Sanity testing is performed on a stable application.

Browser Tests

- Checks a web app for browser compatibility.
- Tests that the app can be easily accessed from all versions of major web browsers.
- Tooling:
 - [CrossBrowserTesting.com](https://crossbrowsertesting.com), LambdaTest, [Browsershots](https://www.browsershots.com), [Experitest](https://www.experitest.com), [Turbo Browser Sandbox](https://www.turbobrowser.com), [Ranorex Studio](https://www.ranorex.com), [Browserstack](https://www.browserstack.com), etc.

End-to-end Tests

- A functional testing of the entire software system.
- You perform fewer end-to-end tests than integration tests.
- Tooling
 - [Cucumber](#), [Protractor](#), [Jasmine](#), [Karma](#), [SpecFlow](#), etc. are some great end-to-end testing tools.

User Acceptance Tests

- Defines how well an application is accepted by real end-users.
- The fulfilment of business requirements, by actual users, is checked, in a real customer- / user environment.
- V-Model location
 - In the top
 - The last level of testing.
 - Runs parallel to the requirement analysis phase.

Non-functional Tests

Non-functional Tests

- Performance
- Stress
- Volume
- Load
- Security
- Scalability
- Usability
- Maintainability
- Compatibility
- Failover
- Compliance
- Efficiency
- Endurance
- Recovery
- Localization
- Internationalization

Performance Tests

- The performance of the application is measured while subjecting the application to real-world conditions.
- Monitoring performance parameters like
 - response time, scalability, stability and efficiency of resource usage etc.
- Useful in finding out when the application will degrade.
- Helps enhance the application design or architecture in a way that reliability and fast response time is ensured.
- Tooling:
- [WebLOAD](#), [LoadView](#), [NeoLoad](#), [LoadNinja](#), [Appvance](#), [LoadRunner](#), [Apache JMeter](#), [Loadster](#), [LoadImpact](#), [Testing Anywhere](#), [SmartMeter.io](#), [Tricentis Flood](#), [Rational Performance Tester](#), [LoadComplete](#), etc.

Stress Tests

- Application is subjected to anomalous conditions, which will not happen under normal circumstances.
- Load on the application is increased to the point that it breaks, and the application behavior is recorded.
- Questions answered via this testing
 - How does the system behave under stressful conditions?
 - If it crashes, will it be able to recover itself and restart?

Volume Tests

- How well the system behaves when a large volume of data and operations are handled by the database.
- Will it be able to store and process a huge volume of data?
- Will there be issues due to the huge data volume?

Load Tests

- The performance of the application under expected load as in the real world is measured.
- All possible loads on the application are simulated and the performance is checked.
- Tooling
 - [LoadRunner](#), [WebLoad](#), [JMeter](#), etc.

Security Tests

- Security of the application from the point of view of the network, data, system and the application is tested.
- Any non-authorized person should not be able to access the application and should not be able to access any confidential data.
- This testing ensures customer reliability and confidence in the application.
- Penetration testing is an example of Security testing.

Scalability Tests

- In case the application needs to be scalable in future, will the architecture and design allow?
- Ex., if we add more servers, have more database transactions, increase the user load in the future, will our application allow that?

Usability Tests

- Deals with the usability of the system from a user's perspective.
- Is the user able to navigate through the application without help?
- Next time, when the user visits the application, is he able to remember the application without help or problems?
- How efficiently is a user able to use the system?
- All these questions are answered via usability testing.

Maintainability Tests

- Maintaining the application for future expansion and requirements is important.
- How well does the application accommodate changes and updates?

Compatibility Tests

- The compatibility of the application is tested with regard to different
 - Operating Systems.
 - Browsers.
 - Hardware.
 - Network capacity.
 - Devices etc.
- The application should be able to perform well on the prioritized environments by the customer.

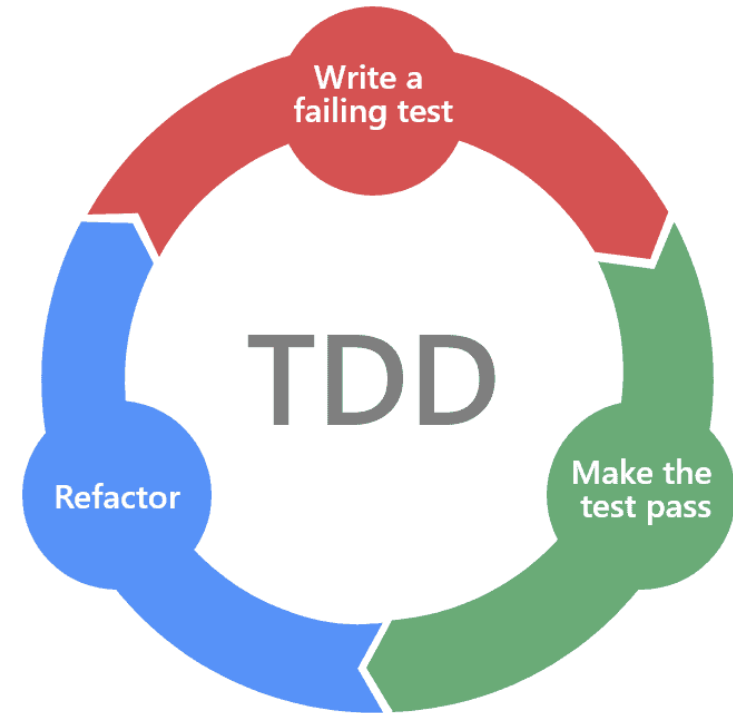
Recovery Tests

- Checking whether the app can recover from crashes and how well it recovers.
- In this kind of tests, testers observe how well the software can come back to the normal flow of execution.
- Crashes can happen anytime. Even if your software is of exceptional quality, crashes may happen. You don't know when they may take place and annoy the users.
- We must implement mechanisms that will recover the app quickly and that will make the app run smoothly again.

Test-Driven-Development (TDD)

Test-Driven-Development (TDD)

- We start with a failing test.
- We create the production code on the fly, while building our test.
- We're done when the test(s) pass.
- Now we refactor, to optimize the test structure and data.



Test Case

- We need a headless (no UI) calculator.
- It gets its data from, some kind of **number source**, which could be different formats, .csv, .xml, .json...
- We
 - Will use JUnit 5 and Mockito Mocks.
 - Don't have a number source implementation yet (using interfaces and mocks).
 - Want to assure, that following criteria are validated
 - $+ * + = +$
 - $- * + = -$
 - $+ * - = -$
 - $- * - = +$

Maven Projects (POMs)

- Properly the most used tool manage complex Java projects.
- Fetches te dependencies (and sub-dependencies) the project depends on, like
 - jar, war, java source, and other build or test artifacts.
- Configured via a pom.xml file in the source code project.
- A POM (Project Object Model) defines project dependencies
- Uses scopes to manage the different groups of dependencies.
- Offers hierarchical structure, where specialized POM can inherit other general POMs.
- Organizations often have / design
 - General POMs for all its systems products, with toolkits and core features.
 - Specialized POM for the individual systems and product variants.

```
<project>
  <modelVersion>1.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>TestDemo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.8.2</version>
      <scope>test</scope>
    </dependency>
    ...
  </dependencies>
</project>
```

Maven Scopes

compile (default scope)

- all dependencies with **runtime**, and **compile** scope dependencies.

provided

- **runtime** and **compile** scope dependencies.

runtime

- **runtime** and **compile** scope transitive dependencies.

test

- **runtime**, **compile** transitive and **test** scope dependencies.

system

- Similar to **provided** except that you have to provide the JAR which contains it explicitly. The artifact is always available and is not looked up in a repository.

import

- Indicates the dependency is to be replaced with the effective list of dependencies in the specified POM's.

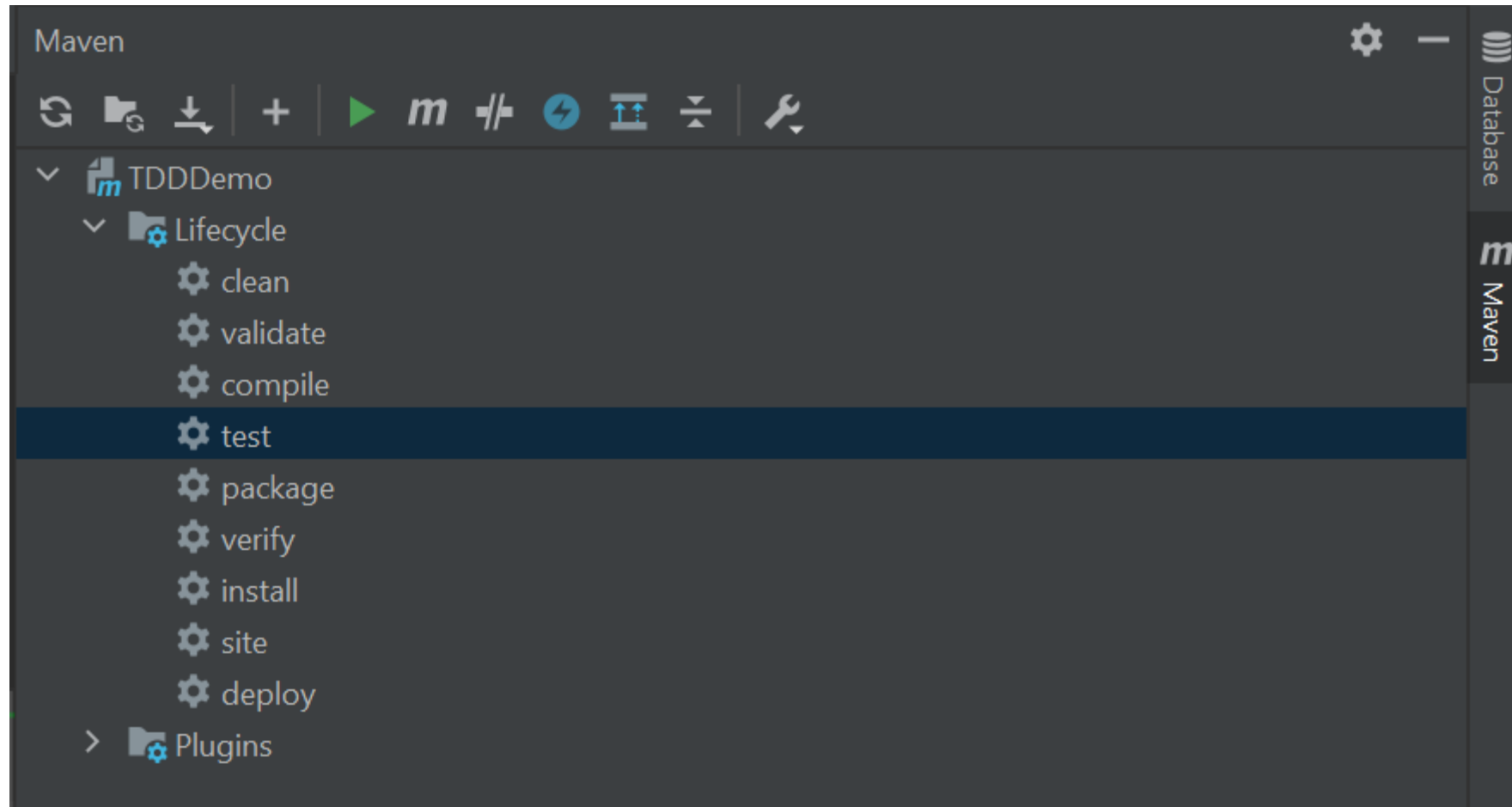
Dependencies with scopes **provided** and **test** will never be included in the main project.

Maven Build Lifecycles

The default lifecycle comprises of the following phases:

- **clean** - cleans the current build folder.
- **validate** - validate the project is correct and all necessary information is available.
- **compile** - compile the source code of the project.
- **test** - test the compiled source code using a suitable unit testing framework.
- **package** - take the compiled code and package it in its distributable format, such as a JAR.
- **verify** - run any checks on results of integration tests to ensure quality criteria are met
- **install** - install the package into a local repository, for use by other projects locally.
- **deploy** - deploys (copies) the final package to the remote repository for it to be shared.

Maven Build Window



Maven Dependencies

JUnit Engine

- groupId = **org.junit.jupiter**
- artifactId = **junit-jupiter-engine**
- version = **5.8.2**
- scope = **test**

JUnit Platform

- groupId = **org.junit.platform**
- artifactId = **junit-platform-runner**
- version = **1.8.2**
- scope = **test**

Mockito (for mocking)

- groupId = **org.mockito**
- artifactId = **mockito-core**
- version = **4.3.1**
- scope = **test**

Maven Dependencies

JUnit Engine

- groupId = **org.junit.jupiter**
- artifactId = **junit-jupiter-engine**
- version = **5.8.2**
- scope = **test**

JUnit Platform

- groupId = **org.junit.platform**
- artifactId = **junit-platform-runner**
- version = **1.8.2**
- scope = **test**

Mockito (for mocking)

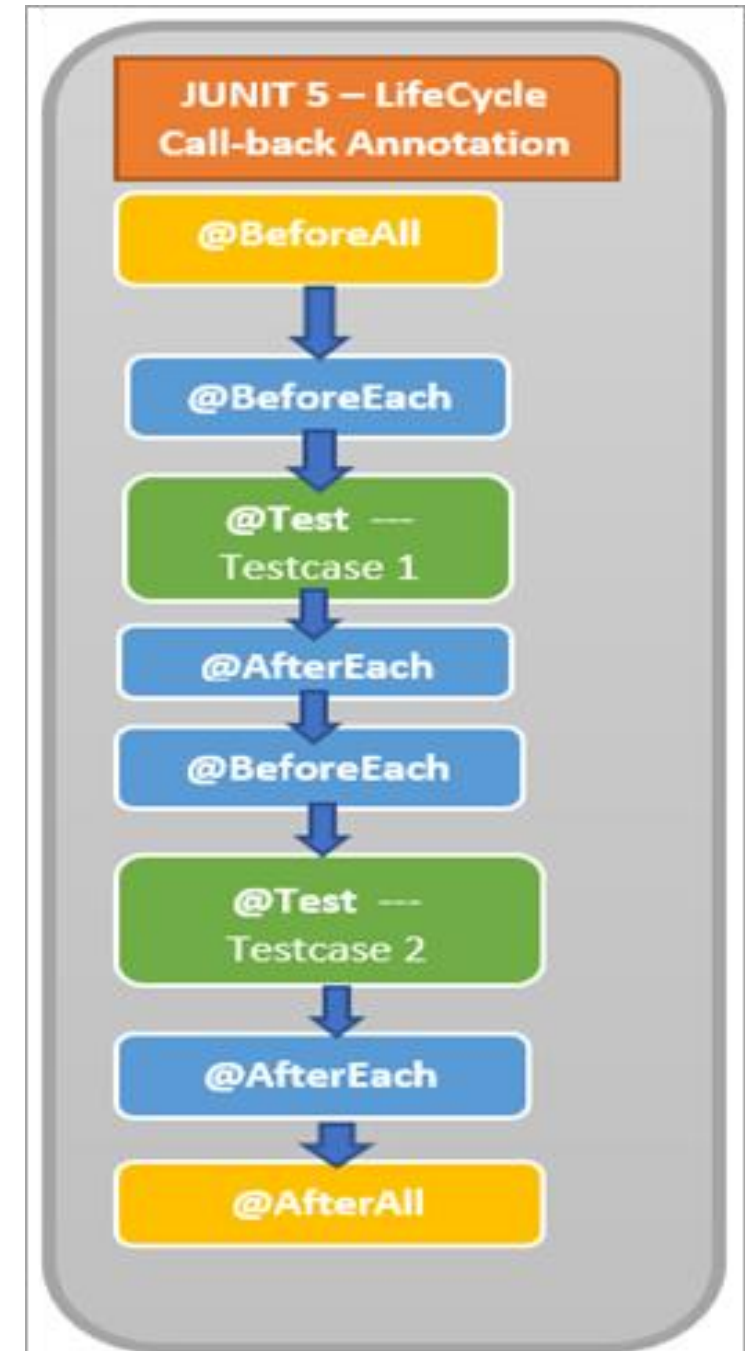
- groupId = **org.mockito**
- artifactId = **mockito-core**
- version = **4.3.1**
- scope = **test**

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.8.2</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.junit.platform</groupId>
    <artifactId>junit-platform-runner</artifactId>
    <version>1.8.2</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>4.3.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

JUnit Annotation Lifecycle



TDD Demo

In Java Maven Projects.

JUnit & Mockito Links

- TDD Demo Tutorial
 - <https://github.com/RonniKahalani/TDDDemo/blob/master/README.md>
- JUnit Example
 - <https://www.journaldev.com/20834/junit5-tutorial>
- Mockito Tutorial
 - <https://www.baeldung.com/mockito-series>

Questions?

Feedback?

Testing types, techniques and tactics

- Installation
- Compatibility
- Smoke and sanity
- Regression
- Acceptance
- Alpha
- Beta
- Functional vs non-functional
- Continuous
- Destructive
- Performance
- Usability
- Accessibility
- Security
- Internationalization and localization
- Development
- A/B
- Concurrency
- Conformance
- Output comparison
- Property
- VCR

Testing Patterns

- Pair testing.
- Separate interface testing
 - For internal classes.
- Scenario testing.